



Recipes & FAQ

Edit

New Page

Lev Konstantinovskiy edited this page on Apr 27 · 54 revisions

Add your useful code snippets and recipes here. You can also post a short question -- please only ask questions that can be fully answered in a sentence or two. No open-ended questions or discussions here.

Q1: How many times does a feature with id 123 appear in a corpus?

A: `total_sum = sum(dict(doc).get(123, 0) for doc in corpus)`

Q2: How do you calculate the vector length of a term?

A: (note that "vector length" only makes sense for non-zero vectors):

1. If the input vector `vec` is in gensim sparse format (a list of 2-tuples): `length = math.sqrt(sum(val**2 for _, val in vec))`, or use `length = gensim.matutils.vecLen(vec)`.
2. If the input vector is a numpy array: `length = gensim.matutils.blas_nrm2(vec)`
3. If the input vector is in a `scipy.sparse` format: `length = numpy.sqrt(numpy.sum(vec.tocsr().data**2))`

Also note that if you want the length just to normalize a vector to unit length, you might as well call `gensim.matutils.unitvec(vec)`, which accepts **any of these three formats** as input.

Q3: How do you calculate the matrix V in LSI space?

A: Given a model `lsi = LsiModel(x, ...)`, with the truncated singular value decomposition of your corpus `x` being `x=U*S*V^T`, doing `lsi[X]` computes `U^-1*x`, which equals `V*S` (basic linear algebra). So if you want `V`, divide `lsi[X]` by `S`:

```
V = gensim.matutils.corpus2dense(lsi[X], len(lsi.projection.s)).T / lsi.projection.s, to get
V as a 2d numpy array.
```

Q4: How do you output the U, S, V^T matrices of LSI?

A: After creating the LSI model `lsi = models.LsiModel(corpus, ...)`, the U and S matrices are in `lsi.projection.u` and `lsi.projection.s`. The V (or `V^T`) matrix is not stored explicitly, because it may not fit in memory (its shape is `num_docs * num_topics`). If you need V, you can compute it with an extra pass over `corpus`, using gensim's streaming `lsi[corpus]` API (see Q3 above).

Q5: I am getting out of memory errors with LSI. How much memory do I need?

A: The final model is stored as a matrix of `num_terms * num_topics` numbers. With 8 bytes per number (double precision), that's `8 * num_terms * num_topics`, i.e. for 100k terms in dictionary and 500 topics, the model will be `8*100,000*500 = 400MB`.

That's just the output -- during the actual computation of this model, temporary copies are needed, so in practice, you'll need about 3x that amount. For the 100k dictionary and 500 topics example, you'll actually need ~1.2GB to create the LSI model.

When out of memory, you'll have to either reduce the dictionary size or the number of topics (or add RAM!). The memory footprint is not affected by the number of training documents, though.

▼ Pages 9

[Home](#)[Developer page](#)[GSOC 2017 project ideas](#)[Hall of WTF](#)[Ideas & Feature proposals](#)[Recipes & FAQ](#)[Roadmap](#)[Student Projects](#)[Word2Vec & Doc2Vec Wishlist](#)

Clone this wiki locally

<https://github.com/RaRe-Tec>[Clone in Desktop](#)

Q6: I have many text files under a directory, each file is a single document. How do I create a corpus from that?

A: See <http://radimrehurek.com/gensim/tut1.html#corpus-streaming-one-document-at-a-time> . If you're having trouble going through the files, have a look at the following snippet (it accepts all .txt files, even in nested subdirectories):

```
def iter_documents(top_directory):
    """Iterate over all documents, yielding a document (=list of utf8 tokens) at a time."""
    for root, dirs, files in os.walk(top_directory):
        for file in filter(lambda file: file.endswith('.txt'), files):
            document = open(os.path.join(root, file)).read() # read the entire document, as
            yield gensim.utils.tokenize(document, lower=True) # or whatever tokenization sui

class MyCorpus(object):
    def __init__(self, top_dir):
        self.top_dir = top_dir
        self.dictionary = gensim.corpora.Dictionary(iter_documents(top_dir))
        self.dictionary.filter_extremes(no_below=1, keep_n=30000) # check API docs for pruni

    def __iter__(self):
        for tokens in iter_documents(self.top_dir):
            yield self.dictionary.doc2bow(tokens)

corpus = MyCorpus('/tmp/test') # create a dictionary
for vector in corpus: # convert each document to a bag-of-word vector
    print vector
...
```

Q7: I have many text files under a directory, each file is a single document. How do I create a word2vec model from that?

A: (by [Christian Ledermann](#))

This code makes the simplifying assumption that sentence-ending punctuation should be excluded from the text and that . and : always end a sentence. [text-sentence text tokenizer and sentence splitter](#) may be a better alternative

```
class DirOfPlainTextCorpus(object):
    """Iterate over sentences of all plaintext files in a directory """
    SPLIT_SENTENCES = re.compile(u"[.!?:]\s+") # split sentences on these characters

    def __init__(self, dirname):
        self.dirname = dirname

    def __iter__(self):
        for fn in os.listdir(self.dirname):
            text = open(os.path.join(self.dirname, fn)).read()
            for sentence in self.SPLIT_SENTENCES.split(text):
                yield gensim.utils.simple_preprocess(sentence, deacc=True)

model = gensim.models.Word2Vec(DirOfPlainTextCorpus('/path/to/dir'), size=200, min_count=5,
```

Q8: How can I filter a saved corpus and its corresponding dictionary?

A: (by [Yaser Martinez](#))

The function `dictionary.filter_extremes` changes the original IDs so we need to reread and (optionally) rewrite the old corpus using a transformation:

```
import copy
from gensim.models import VocabTransform

# filter the dictionary
old_dict = corpora.Dictionary.load('old.dict')
new_dict = copy.deepcopy(old_dict)
new_dict.filter_extremes(keep_n=100000)
new_dict.save('filtered.dict')
```

```
# now transform the corpus
corpus = corpora.MmCorpus('corpus.mm')
old2new = {old_dict.token2id[token]:new_id for new_id, token in new_dict.iteritems()}
vt = VocabTransform(old2new)
corpora.MmCorpus.serialize('filtered_corpus.mm', vt[corpus], id2word=new_dict)
```

Q9: How do I load a model in Python 3 that was trained and saved using Python 2?

A:

This has been [fixed](#) in 0.13.4 release

If you are using an earlier version, read the solution below.

(by [Matti Lyra](#))

Python pickling is not backward compatible. There are two things standing in your way:

1. Python dictionary pickling `LdaModel.id2word`
2. Small NumPy arrays

The `LdaModel.save` already does save large NumPy arrays (> 10MB) to separate files using NumPy's own IO functionality. Smaller arrays however - resulting either from small training corpora or by using `alpha=auto` - are pickled along with the rest of the `LdaModel` object.

Trying to load a model in Python 3 will result in an error like the following

```
UnicodeDecodeError: 'ascii' codec can't decode byte 0xba in position 6: ordinal not in range(128)
```



To get around this you need to remove the `id2word` dictionary from the `LdaModel` before saving, and ensure that *all* the NumPy arrays regardless of size are saved to separate files.

```
# Python 2
import json
from gensim.models.ldamodel import LdaModel

id2word = {k:v for k, v in lda.id2word.items()}
lda.id2word = None
# save the expElogbeta and state.sstats separately using numpy not pickle,
# if you're using alpha=auto or you've set alpha or eta to some array yourself you
# should add 'alpha', and 'eta' to the 'separately' list
lda.save('~/Desktop/temp/migrate.2to3.gensim', separately=['expElogbeta', 'sstats'])
lda.id2word = id2word # restore the dictionary

with open('~/Desktop/temp/migrate.2to3.id2word.json', 'wb') as out:
    json.dump(id2word, out)
```

You can then load the model normally in Python 3, but have to remember to also load the dictionary:

```
# Python 3
import json
from gensim.models.ldamodel import LdaModel

with open('~/Desktop/temp/migrate.2to3.id2word.json') as fh:
    id2word = json.load(fh)
id2word = {int(k):v for k, v in id2word.items()}

# load the model and replace the separately stored id2word dictionary
lda = LdaModel.load('~/Desktop/temp/migrate.2to3.gensim')
lda.id2word = id2word
```

Q10: Loading a word2vec model fails with `UnicodeDecodeError: 'utf-8' codec can't decode bytes in position ...`

A: The strings (words) stored in your model are not valid utf8. By default, gensim decodes the words using the `strict` encoding settings, which results in the above exception whenever an invalid utf8 sequence is encountered.

The fix is on your side and it is to either:

a) Store your model using a program that understands unicode and utf8 (such as gensim). Some C and Java word2vec tools are known to **truncate the strings at *byte* boundaries**, which can result in cutting a multi-byte utf8 character in half, making it non-valid utf8, leading to this error.

b) Set the `unicode_errors` flag when running `load_word2vec_model`, e.g. `load_word2vec_model(..., unicode_errors='ignore')`. Note that this silences the error, but the utf8 problem is still there -- invalid utf8 characters will just be ignored in this case.

Q11:How can I see the actual words rather than an integer that represents the word? Basically, how do I know what integer represents what words?

